SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1710**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Deepika S / 1921260057 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. **A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:**

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:

Problem understanding (CSP formulation)

- Variables: D1, D2, D3 (the shift of each doctor).
- Domain of each variable: {Morning (M), Afternoon (A), Night (N)}.
- Constraints: C1: D1 ≠ N. C2: shift(D2) < shift(D3). C3: D1, D2, D3 take different shifts (one doctor per shift). C4: D3 ≠ M. C5: every doctor gets exactly one shift (satisfied because each variable receives one value).

Method

Backtracking search assigns one variable at a time (order D1, D2, D3; values tried in the order M, A, N), checks the constraints after every assignment, and goes back to try the next value when a constraint fails.

Steps and constraint checks

Step	Assignment tried	Constraint checks	Result
1	D1 = M	C1: M ≠ N ✓. C3: no other doctor assigned ✓	Accept
2	D2 = M	C3: Morning already used by D1 ✗	Reject, next value
3	D2 = A	C3: Afternoon free ✓. C2 cannot be checked yet (D3 unassigned)	Accept
4	D3 = M	C4: D3 cannot work Morning ✗. C3: Morning used ✗	Reject, next value
5	D3 = A	C3: Afternoon used by D2 ✗. C2: A < A is false ✗	Reject, next value
6	D3 = N	C4: N ≠ M ✓. C3: Night free ✓. C2: D2 = A < D3 = N ✓. C1 ✓	Accept, all assigned

Final valid schedule

Shift	Morning	Afternoon	Night
Doctor	D1	D2	D3

Interpretation

All variables are assigned and every constraint is satisfied, so the search stops without backtracking to D1. The schedule D1 – Morning, D2 – Afternoon, D3 – Night satisfies all five constraints. Had the search continued, it would also find the second valid schedule D2 – Morning, D1 – Afternoon, D3 – Night; the first solution found is returned.

2. **A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.**

   Constraints:

   i. Movement allowed: Up, Down, Left, Right

   ii. Cost of each move = 1

   iii. Cannot pass through obstacles

   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Answer:

Problem understanding

The grid was not drawn in the question, so the following 5 × 5 grid is assumed. Cells are written as (row, column) with (0,0) at the top left. Start S = (0,0), Goal G = (4,4). Obstacles (shaded) = (1,1), (2,1), (1,3), (3,3).

S	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)		(1,2)		(1,4)
(2,0)		(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	(3,2)		(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	**G**

Manhattan distance to the goal: h(r, c) = |4 − r| + |4 − c|, so h(S) = 8. Every move costs 1, so g(n) equals the number of moves from S.

Method

Breadth-First Search (BFS) expands nodes level by level using a FIFO queue. With every move costing 1, BFS finds the minimum-cost path. Neighbours are generated in the order Up, Down, Left, Right, and a cell is marked visited when it is added to the queue. The same grid is then solved with the Manhattan-distance priority queue (ordered by f = g + h) to show how the heuristic reduces the search.

(A) BFS: node expansion and queue updates

Step	Node expanded	Neighbours added	Queue after step (front → back)
1	(0,0) g=0	(1,0), (0,1)	(1,0), (0,1)
2	(1,0) g=1	(2,0) [(1,1) obstacle]	(0,1), (2,0)
3	(0,1) g=1	(0,2) [(1,1) obstacle]	(2,0), (0,2)

4	(2,0) g=2	(3,0) [(2,1) obstacle]	(0,2), (3,0)
5	(0,2) g=2	(1,2), (0,3)	(3,0), (1,2), (0,3)
6	(3,0) g=3	(4,0), (3,1)	(1,2), (0,3), (4,0), (3,1)
7	(1,2) g=3	(2,2) [(1,1), (1,3) obstacles]	(0,3), (4,0), (3,1), (2,2)
8	(0,3) g=3	(0,4) [(1,3) obstacle]	(4,0), (3,1), (2,2), (0,4)
9	(4,0) g=4	(4,1)	(3,1), (2,2), (0,4), (4,1)
10	(3,1) g=4	(3,2)	(2,2), (0,4), (4,1), (3,2)
11	(2,2) g=4	(2,3)	(0,4), (4,1), (3,2), (2,3)
12	(0,4) g=4	(1,4)	(4,1), (3,2), (2,3), (1,4)
13	(4,1) g=5	(4,2)	(3,2), (2,3), (1,4), (4,2)
14	(3,2) g=5	none new [(3,3) obstacle]	(2,3), (1,4), (4,2)
15	(2,3) g=5	(2,4)	(1,4), (4,2), (2,4)
16	(1,4) g=5	none new	(4,2), (2,4)
17	(4,2) g=6	(4,3)	(2,4), (4,3)
18	(2,4) g=6	(3,4)	(4,3), (3,4)
19	(4,3) g=7	(4,4) = G	(3,4), (4,4)

G = (4,4) is reached at depth 8 (after 19 expansions). Following the parent links back from G: (4,4) ← (4,3) ← (4,2) ← (4,1) ← (4,0) ← (3,0) ← (2,0) ← (1,0) ← (0,0).

(B) Priority queue with Manhattan distance (f = g + h)

Ties in f are broken by the smaller h. Each entry is written as cell [g + h = f].

Step	**Node expanded**	**Successors added**	**Priority queue after step**
1	(0,0) [0+8=8]	(1,0) [1+7=8], (0,1) [1+7=8]	(1,0) 8, (0,1) 8
2	(1,0) [1+7=8]	(2,0) [2+6=8]	(2,0) 8, (0,1) 8
3	(2,0) [2+6=8]	(3,0) [3+5=8]	(3,0) 8, (0,1) 8
4	(3,0) [3+5=8]	(4,0) [4+4=8], (3,1) [4+4=8]	(4,0) 8, (3,1) 8, (0,1) 8
5	(4,0) [4+4=8]	(4,1) [5+3=8]	(4,1) 8, (3,1) 8, (0,1) 8
6	(4,1) [5+3=8]	(4,2) [6+2=8]	(4,2) 8, (3,1) 8, (0,1) 8
7	(4,2) [6+2=8]	(4,3) [7+1=8], (3,2) [7+3=10]	(4,3) 8, (3,1) 8, (0,1) 8, (3,2) 10
8	(4,3) [7+1=8]	(4,4) [8+0=8]	(4,4) 8, (3,1) 8, (0,1) 8, (3,2) 10
9	(4,4) = G	Goal removed from queue: stop	—

Optimal path and cost

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4), i.e. Down 4 times, then Right 4 times. Number of moves = 8, so the total cost = 8 × 1 = 8.

Interpretation

The cost 8 equals the Manhattan distance from S to G, so no shorter path can exist and the path is optimal. BFS needed 19 expansions to find it, whereas the Manhattan-distance priority queue needed only 9, because it never wandered into cells with a larger f value.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- **The robot can move up, down, left, right**
- **Some cells are blocked (obstacles) and cannot be traversed**
- **The robot has limited sensing capability (can only observe adjacent cells)**
- **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- **The robot must minimize total cost while ensuring safe navigation**
- **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

- **Intelligent agents**
- **Rationality**
- **Environment type**

Answer:

a) Analysis of the constraints

Constraint	**Effect on decision making**
Moves: up, down, left, right	State is the robot's cell; at most 4 actions per state, so the branching factor is small
Blocked cells	Hard constraint: such cells are never entered. A blocked cell found by sensing makes the current plan invalid, so the robot must re-plan
Sensing limited to adjacent cells	The environment is partially observable. The full map is unknown in advance, so the robot cannot compute a complete path offline; it must explore and remember what it sees
Move cost 1, risky cell +2	Entering a risky cell costs 1 + 2 = 3. The path with the fewest moves is not always the cheapest, so costs must be compared (BFS is not suitable)
Minimize cost with safe navigation	Safety is a priority: a risky cell is entered only when every safe route costs more, and a tie is settled in favour of the safe route
Dynamic knowledge update	Planning and acting are interleaved (online search): after every move the robot updates its map and re-plans if needed

Priority of constraints: (1) blocked cells and safety (must never be violated), (2) limited sensing (decides how the robot plans), (3) minimum cost (optimized among the safe and feasible choices).

b) Solution approach: Online search agent with Uniform Cost Search

The robot acts as an online search agent that runs Uniform Cost Search (UCS) on the part of the map it currently knows, takes one step, senses, and repeats.

1. Initialize the knowledge map: the robot's own cell is known; all other cells are assumed free (cost 1) until sensed.
2. Sense the four adjacent cells and record each as free, blocked, risky or survivor. If G is seen, mark it as the goal.
3. Run UCS from the current cell to G on the known map. Step cost = 1 for a free cell and 1 + 2 = 3 for a risky cell; blocked cells are removed. UCS always expands the lowest path-cost node from a priority queue.
4. If UCS returns a path, execute only its first move and add the cost of that move to the total.
5. At the new cell, sense the neighbours again and update the map.
6. If a newly found blocked or risky cell affects the planned path, discard the old plan and go back to step 3 (re-plan).
7. Keep a record of visited cells so that the robot does not loop; if UCS finds no path, return to the last junction with an unexplored neighbour and explore from there.
8. Stop when the robot reaches G (survivor located) and report the total cost.

Pseudocode

```
function ONLINE-UCS-AGENT(percept) returns action
  persistent: map, position, total_cost
  map ← UPDATE-MAP(map, percept)              // adjacent cells: free / blocked / risky / goal
  if position = GOAL then return STOP
  path ← UCS(map, position, GOAL)             // step cost: 1 (free), 3 (risky)
  if path = failure then return BACKTRACK-TO-UNEXPLORED(map, position)
  action ← FIRST-MOVE(path)
  total_cost ← total_cost + STEP-COST(map, action)
  position ← RESULT(position, action)
  return action
```

c) Application of AI concepts

Intelligent agent. The robot perceives its environment through sensors and acts on it through actuators (wheels). Its PEAS description is:

Component	Description
Performance measure	Reach the survivor, minimum total path cost, no collision and minimum entry into risky zones
Environment	Grid of rooms with unknown obstacles and risky zones
Actuators	Move up, move down, move left, move right, stop at G
Sensors	Adjacent-cell sensor (free / blocked / risky / survivor), position sensor

The agent is goal-based (goal: reach G) and utility-based (it prefers the lowest-cost, safest route). It is also model-based, because it keeps an internal map built from past percepts.

Rationality. A rational agent selects the action that is expected to maximize its performance measure, given its percept history and built-in knowledge. Here, the robot always follows the cheapest path on its current map and re-plans when it learns something new. Example: entering a risky cell costs 3 instead of 1, an extra 2. If a safe detour adds 1 extra move, the robot takes the detour; if the detour adds 2 extra moves, the costs tie and the safe route is chosen; if the detour adds 3 or more extra moves, entering the risky cell is the cheaper option.

Environment type.

Property	**Type and reason**
Observability	Partially observable: only adjacent cells are sensed
Agents	Single agent (the rescue robot)
Determinism	Deterministic moves, but the map is unknown and uncertain (treated as stochastic in practice because of debris)
Episodic / Sequential	Sequential: each move affects later cost and position
Static / Dynamic	Semi-dynamic: the building may change (collapse), and the robot's knowledge changes with every move
Discrete / Continuous	Discrete: grid cells and fixed moves
Known / Unknown	Unknown: the layout is learned during exploration

d) Justification of the chosen approach

- The map is unknown and only adjacent cells are visible, so a complete plan cannot be made before moving. An online search agent that interleaves planning and acting is therefore required.
- Move costs are not equal (1 for a normal cell, 3 for a risky cell). BFS minimizes only the number of moves and DFS gives no cost guarantee, whereas UCS always expands the cheapest path first and is complete and optimal when all step costs are positive.
- Re-planning after each sensing step lets the robot adjust at once when a blocked or risky cell is discovered, so unsafe cells are not entered.
- The robot builds its map as it moves, which avoids repeated exploration and loops.
- UCS needs no heuristic, which suits a case where the position of G may be unknown at the start. Once G is seen, the Manhattan distance (admissible, since the minimum step cost is 1) can be added to turn UCS into A* and reduce the nodes expanded.

Interpretation

The online UCS agent gives the lowest total cost found so far on the known map at every step, keeps the robot away from obstacles and, where possible, from risky zones, and finally reaches the survivor at G.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided